

MOUNTING AND DESTROYING THE VIRTUAL DOM

Created: 2025-12-04 Thu 13:39

RECAPITULATION OF PROGRESS SO FAR

RECAPITULATION OF PROGRESS SO FAR

- learned about Virtual DOM
 - how to create it

RECAPITULATION OF PROGRESS SO FAR

- learned about Virtual DOM
 - how to create it
- implemented:
 - `h ( )` - creates *vnodes* for elements.
 - `hString ( )` - creates *text* nodes.
 - `hFragment ( )` - creates *fragment* nodes.
 - defined `DOM_TYPES` for the different types of *vnodes*.

WHAT WE WILL COVER IN THIS LECTURE

WHAT WE WILL COVER IN THIS LECTURE

- Creating HTML nodes from virtual DOM nodes

WHAT WE WILL COVER IN THIS LECTURE

- Creating HTML nodes from virtual DOM nodes
- Inserting HTML nodes into the browser's document

WHAT WE WILL COVER IN THIS LECTURE

- Creating HTML nodes from virtual DOM nodes
- Inserting HTML nodes into the browser's document
- Removing HTML nodes from the browser's document

MOUNTING THE VIRTUAL DOM

MOUNTING THE VIRTUAL DOM

- Create a real DOM tree for a given virtual DOM tree

MOUNTING THE VIRTUAL DOM

- Create a real DOM tree for a given virtual DOM tree
- Attach it to the browser's document

MOUNTING THE VIRTUAL DOM

- Create a real DOM tree for a given virtual DOM tree
- Attach it to the browser's document
- This should be done in the framework code
 - Users shouldn't use the *Document API*

THE `ReactDOM()` FUNCTION

- We need to create a mount function - `ReactDOM()`.

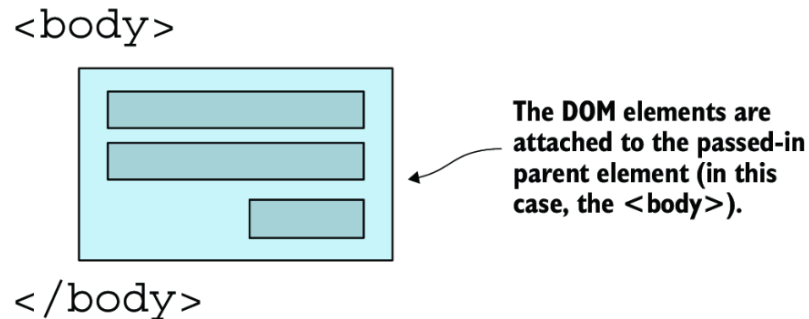
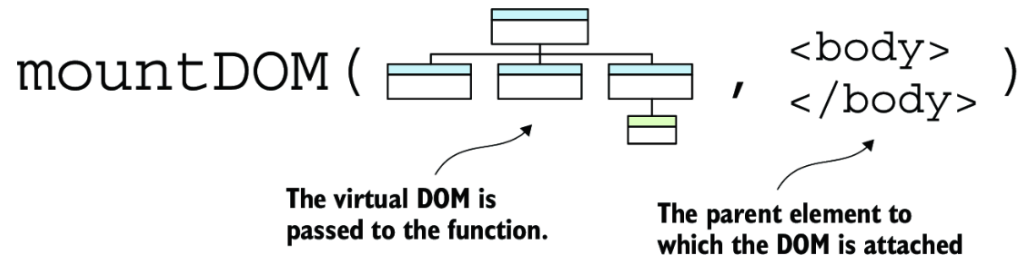


Figure 1: Visual representation of the mountDOM() function

- The first argument is the *virtual DOM*.

- The first argument is the *virtual DOM*.
- Second argument is the parent element where the view will be inserted

- The first argument is the *virtual DOM*.
- Second argument is the parent element where the view will be inserted

*The result is a DOM tree that is attached to the parent element (e.g. **body** element)*

REFERENCES

REFERENCES

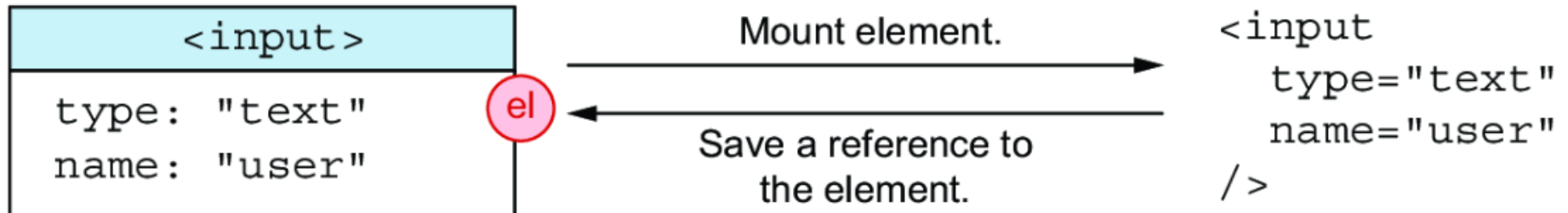
- Furthermore, a reference (connection) needs to be stored to the real DOM node for better optimization.

REFERENCES

- Furthermore, a reference (connection) needs to be stored to the real DOM node for better optimization.
- add an `e1` property in the *vNode*.

REFERENCES

- Furthermore, a reference (connection) needs to be stored to the real DOM node for better optimization.
- add an `el` property in the *vNode*.



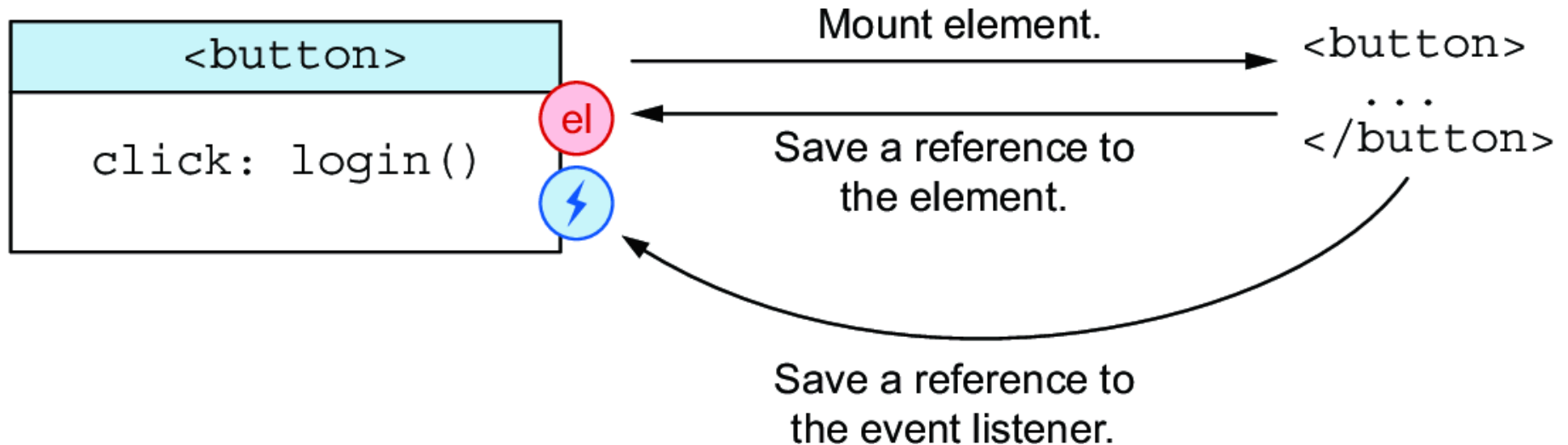
EVENT LISTENERS

EVENT LISTENERS

- Similarity, event listeners can be saved and referenced under the `listeners` property

EVENT LISTENERS

- Similarity, event listeners can be saved and referenced under the `listeners` property



REFERENCES - PURPOSE

Dual use for the references:

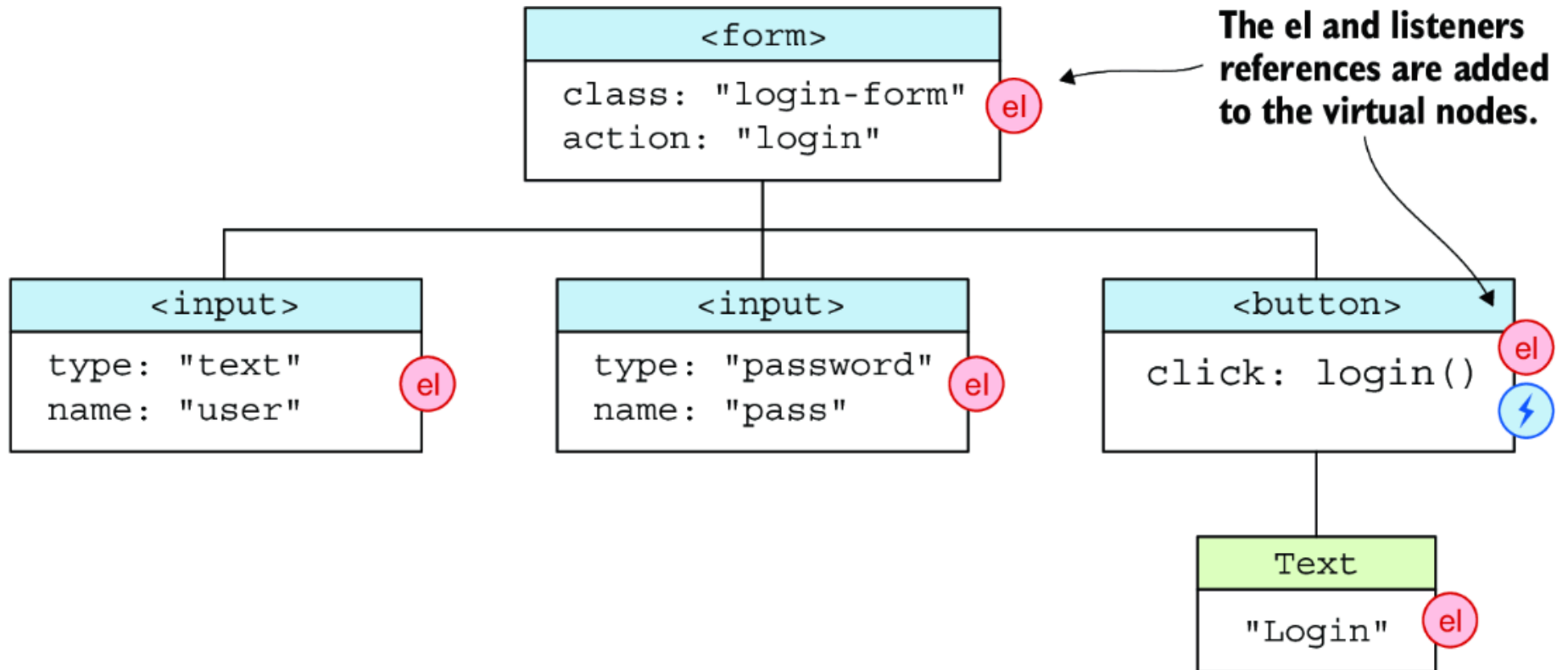
- remove event listeners and detach elements from DOM when the *vnode* is **unmounted**.
- Know what element needs to be updated for the reconciliation algorithm.

OUR EXAMPLE:

```
const vdom = h('form', { class: 'login-form', action: 'login' }, [  
  h('input', { type: 'text', name: 'user' }),  
  h('input', { type: 'password', name: 'pass' }),  
  h('button', { on: { click: login } }, ['Login'])  
)
```

passed to the mountDOM ()

```
mountDOM(vdom, document.body)
```



- This will be attached to the `<body>` element:

- This will be attached to the `<body>` element:

```
<body>
  <form class="login-form" action="login">
    <input type="text" name="user">
    <input type="password" name="pass">
    <button>Login</button>
  </form>
</body>
```

TYPES OF NODES CREATED:

- A virtual node of type `text` requires a `Text` node to be created (via the `document.createTextNode()` method).
- A virtual node of type `element` requires an `Element` node to be created (via the `document.createElement()` method).

MOUNTING VIRTUAL NODES INTO THE DOM

- Let's create the mountDOM function inside `src/mount-dom.js`:

```
// FILE: src/mount-dom.js
import { DOM_TYPES } from './h'

export function mountDOM(vdom, parentEl) {
  switch (vdom.type) {
    case DOM_TYPES.TEXT: { // Mounts a text vnode
      createTextNode(vdom, parentEl)
      break
    }
    case DOM_TYPES.ELEMENT: { // Mounts a element vnode
      createElementNode(vdom, parentEl)
      break
    }
  }
}
```



```
// Continued from previous slide...
case DOM_TYPES.FRAGMENT: { // Mounts a fragment vnode
    createFragmentNodes(vdom, parentEl)
    break
}

default: {
    throw new Error(`Can't mount DOM of type: ${vdom.type}`)
}
}
```

MOUNTING TEXT NODES

```
{  
  type: DOM_TYPES.TEXT,  
  value: 'I need more coffee'  
}
```

MOUNTING TEXT NODES

- use the provided `createTextNode()` DOM method.

```
{  
  type: DOM_TYPES.TEXT,  
  value: 'I need more coffee'  
}
```

MOUNTING TEXT NODES

- use the provided `createTextNode()` DOM method.
- recall the `hString()` function

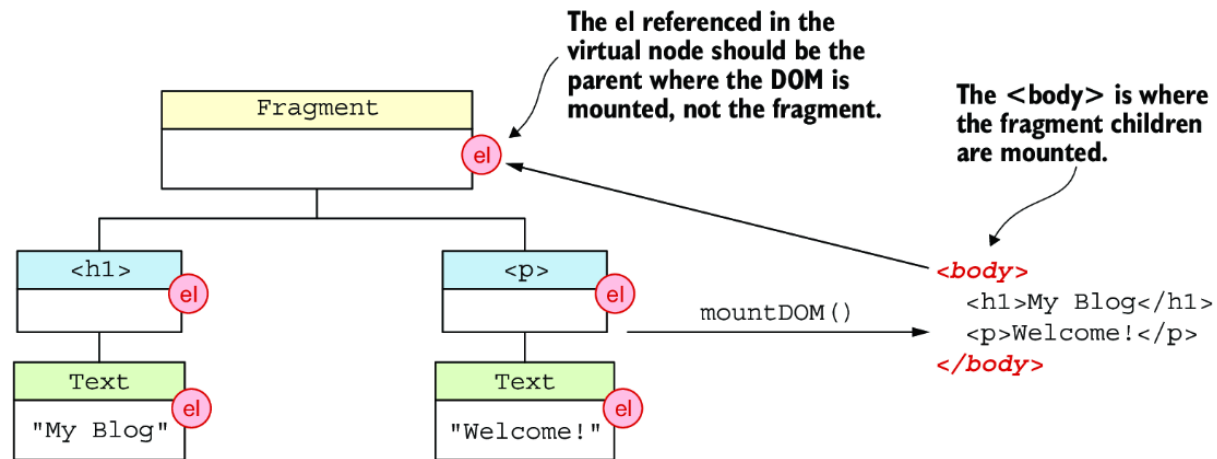
```
{  
  type: DOM_TYPES.TEXT,  
  value: 'I need more coffee'  
}
```

```
// FILE: src/mount-dom.js
function createTextNode(vdom, parentEl) {
  const { value } = vdom

  const textNode = document.createTextNode(value) // Creates text node
  vdom.el = textNode // Saves a reference

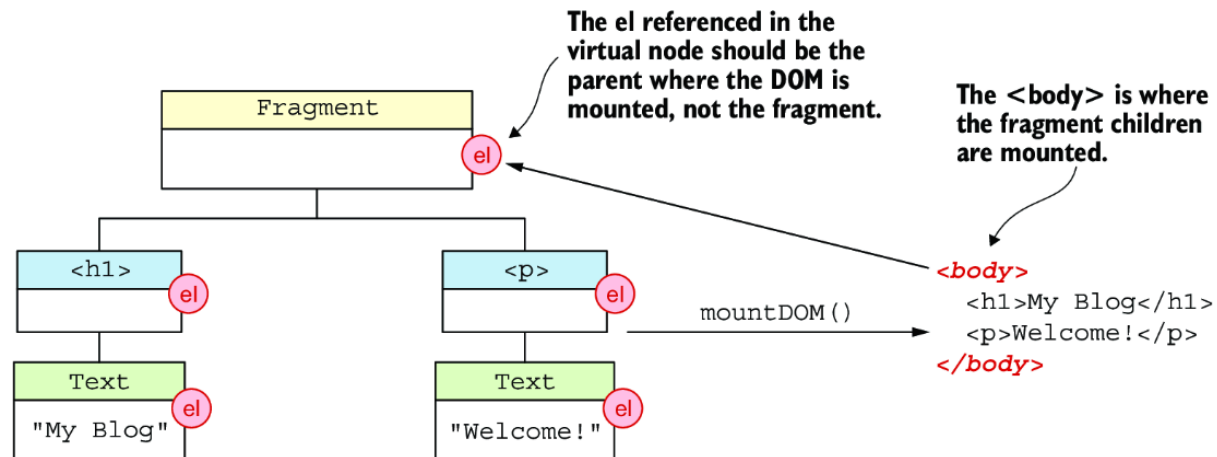
  parentEl.append(textNode) // Appends to the parent element
}
```

MOUNTING FRAGMENT NODES



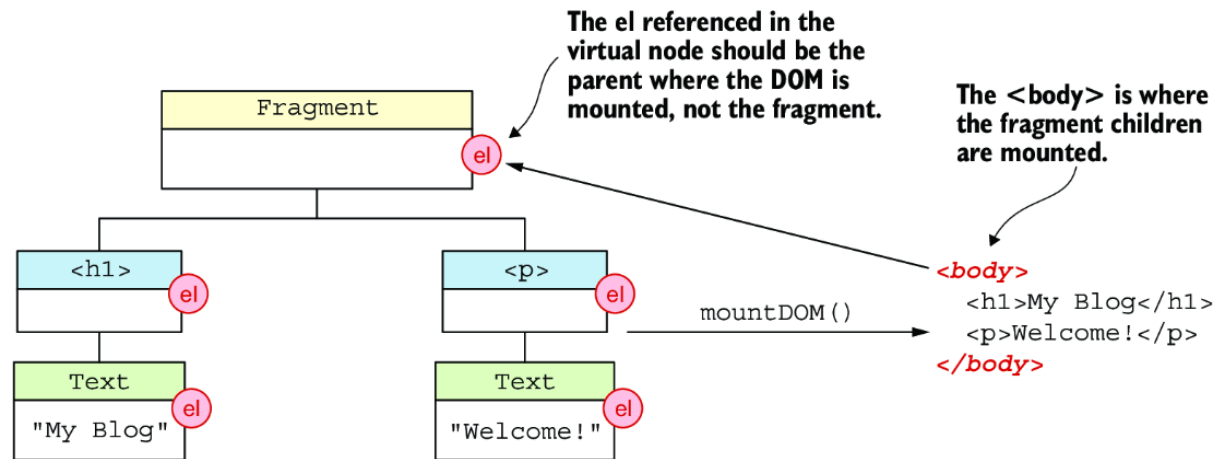
MOUNTING FRAGMENT NODES

- mount the children of the fragment

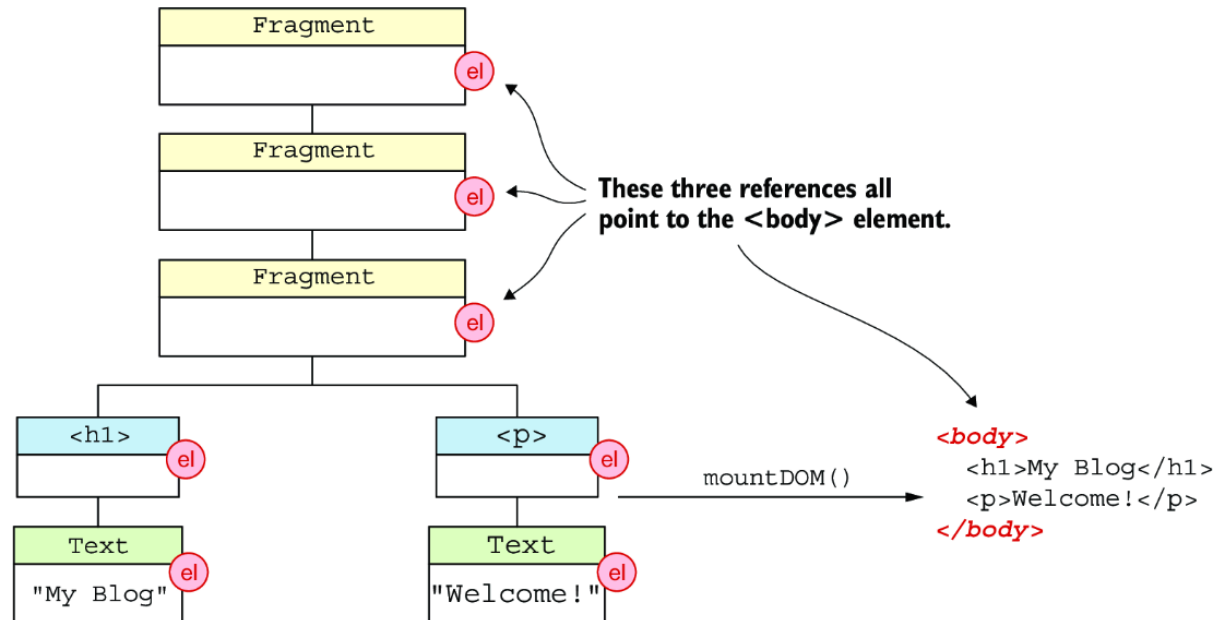


MOUNTING FRAGMENT NODES

- mount the children of the fragment
- reminder: fragment nodes are not attached to the DOM
 - the `el` property of a fragment *vnode* references the parent element



- Nested fragment nodes should have the `el` property referencing the parent element



CreateFragmentNode FUNCTION

```
function createFragmentNodes(vdom, parentEl) {  
  const { children } = vdom  
  vdom.el = parentEl // Saves a reference to the parent element  
  
  // Append each child to the parent element  
  children.forEach((child) => mountDOM(child, parentEl))  
}
```

MOUNTING ELEMENT NODES

- use `createElement ( )` method from the *Document API*.
- The *Document API* returns either:
 - element node that matches the tag name
 - `HTMLUnknownElement` if the tag is unrecognized

- Recall the `h ( )` function result

```
{ type: DOM_TYPES.ELEMENT,  
  tag: 'button',  
  props: {  
    class: 'btn',  
    on: { click: () => console.log('yay!') }  
  },  
  children: [  
    {  
      type: DOM_TYPES.TEXT,  
      value: 'Click me!'  
    }  
  ]  
}
```

CREATION OF A DOM ELEMENT FROM A *VNODE*:

CREATION OF A DOM ELEMENT FROM A *VNODE*:

1. Create the element node using the `document.createElement()` function.

CREATION OF A DOM ELEMENT FROM A *VNODE*:

1. Create the element node using the `document.createElement()` function.
2. Add the attributes and event listeners to the element node, saving the added event listeners in a new property of the virtual node called `listeners`.

CREATION OF A DOM ELEMENT FROM A *VNODE*:

1. Create the element node using the `document.createElement()` function.
2. Add the attributes and event listeners to the element node, saving the added event listeners in a new property of the virtual node called `listeners`.
3. Save a reference to the element node in the virtual node under the `el` property.

CREATION OF A DOM ELEMENT FROM A *VNODE*:

1. Create the element node using the `document.createElement()` function.
2. Add the attributes and event listeners to the element node, saving the added event listeners in a new property of the virtual node called `listeners`.
3. Save a reference to the element node in the virtual node under the `el` property.
4. Mount the children recursively into the element node.

CREATION OF A DOM ELEMENT FROM A *VNODE*:

1. Create the element node using the `document.createElement()` function.
2. Add the attributes and event listeners to the element node, saving the added event listeners in a new property of the virtual node called `listeners`.
3. Save a reference to the element node in the virtual node under the `e1` property.
4. Mount the children recursively into the element node.
5. Append the element node to the parent element.

THINGS TO CONSIDER

- `props` holds both `attributes` and `event listeners`
- they should be handled separately
- `style` and `class` attributes should also be treated separately

WRITING THE FUNCTION

```
// FILE: src/mount-dom.js
import { setAttributes } from './attributes'
import { addEventListeners } from './events'
// Other functions go here...

function createElementNode(vdom, parentEl) {
  const { tag, props, children } = vdom

  // Creates element node
  const element = document.createElement(tag)

  // Adds attributes and event listeners
  addProps(element, props, vdom)
  vdom.el = element

  children.forEach((child) => mountDOM(child, element))
  parentEl.append(element)
}
```

```
// Continued from previous slide...
function addProps(el, props, vdom) {
  // Splits listeners from attributes
  const { on: events, ...attrs } = props

  // Adds event listeners
  vdom.listeners = addEventListeners(events, el) // TODO: implement
  // Sets attributes
  setAttributes(el, attrs) // TODO: implement
}
```

addEventListener () FUNCTION

addEventListener() FUNCTION

- use the `addEventListener()` method from the DOM

addEventListener() FUNCTION

- use the `addEventListener()` method from the DOM
- Elements implement the `EventTarget` interface.

addEventListener() FUNCTION

- use the `addEventListener()` method from the DOM
- Elements implement the `EventTarget` interface.
- create an `addEventListener()` function to do it.

addEventListener() FUNCTION

- use the `addEventListener()` method from the DOM
- Elements implement the `EventTarget` interface.
- create an `addEventListener()` function to do it.

```
// FILE: src/event.js
export function addEventListener(eventName, handler, el) {
  el.addEventListener(eventName, handler)
  return handler
}
```


The vdom example should be:

```
{
  type: DOM_TYPES.ELEMENT,
  tag: 'button',
  props: {
    on: {
      mouseover: () => console.log('almost yay!'),
      click: () => console.log('yay!') ,
      dblclick: () => console.log('double yay!'),
    }
  }
}
```

Implementation of addEventListeners() function

```
export function addEventListeners(listeners = {}, el) {  
  const addedListeners = {}  
  
  Object.entries(listeners).forEach(([eventName, handler]) => {  
    const listener = addEventListener(eventName, handler, el)  
    addedListeners[eventName] = listener  
  })  
  return addedListeners  
}
```

SETTING THE ATTRIBUTES

SETTING THE ATTRIBUTES

- Lets look how attributes are represented into the DOM

SETTING THE ATTRIBUTES

- Lets look how attributes are represented into the DOM

```
<p id="foo">Hello, world!</p>
```

SETTING THE ATTRIBUTES

- Lets look how attributes are represented into the DOM

```
<p id="foo">Hello, world!</p>
```

- If the `<p>` element is created, the `id` will be set as:

SETTING THE ATTRIBUTES

- Lets look how attributes are represented into the DOM

```
<p id="foo">Hello, world!</p>
```

- If the `<p>` element is created, the `id` will be set as:

```
p.id = 'foo'
```

- Some edge-cases:

- Some edge-cases:
 - the `value` attribute of an `<input>` isn't reflected in the rendered HTML

- Some edge-cases:
 - the `value` attribute of an `<input>` isn't reflected in the rendered HTML
 - `class` and `style` attributes should be set using special functions:

```
// FILE: src/attributes.js
export function setAttributes(el, attrs) {
  // Split the attributes
  const { class: className, style, ...otherAttrs } = attrs
  // Set the class attributes
  if (className) {
    // TODO: implement setClass
    setClass(el, className)
  }
  // Sets the style attributes
  if (style) {
    Object.entries(style).forEach(([prop, value]) => {
      // TODO: implement setStyle
      setStyle(el, prop, value)
    })
  }
  // Set all of the other attributes
  for (const [name, value] of Object.entries(otherAttrs)) {
    // TODO: implement setAttributes
    setAttribute(el, name, value)
  }
}
```

SETTING THE CLASS ATTRIBUTE

SETTING THE CLASS ATTRIBUTE

- DOM elements has the `className` and `classList` property

SETTING THE CLASS ATTRIBUTE

- DOM elements has the `className` and `classList` property
 - `className` takes a string of all the class names

SETTING THE CLASS ATTRIBUTE

- DOM elements has the `className` and `classList` property
 - `className` takes a string of all the class names
 - `classList` takes a list of all the classes

SETTING THE CLASS ATTRIBUTE

- DOM elements has the `className` and `classList` property
 - `className` takes a string of all the class names
 - `classList` takes a list of all the classes
 - Both approaches are valid


```
<div class="foo bar baz"></div>
```

- Usage of `classList`:

```
<div class="foo bar baz"></div>
```

- Usage of `classList`:

```
<div></div>
```

```
<div class="foo bar baz"></div>
```

- Usage of `classList`:

```
<div></div>
```

- Adding the classes `foo`, `bar` & `baz`

```
<div class="foo bar baz"></div>
```

- Usage of `classList`:

```
<div></div>
```

- Adding the classes `foo`, `bar` & `baz`

```
div.classList.add('foo', 'bar', 'baz')
```

```
<div class="foo bar baz"></div>
```

- Usage of `classList`:

```
<div></div>
```

- Adding the classes `foo`, `bar` & `baz`

```
div.classList.add('foo', 'bar', 'baz')
```

-The end result

```
<div class="foo bar baz"></div>
```

- Usage of `className`

```
div.className = 'foo bar baz'
```

Possible /vnode/s

```
{  
  type: DOM_TYPES.ELEMENT,  
  tag: 'div',  
  props: {  
    class: ['foo', 'bar', 'baz']  
  }  
}
```

or

```
{  
  type: DOM_TYPES.ELEMENT,  
  tag: 'div',  
  props: {  
    class: 'foo bar baz'  
  }  
}
```


Implementation of setClass function

```
// FILE: src/attributes.js
function setClass(el, className) {
  // Clear the class attribute
  el.className = ''

  // Sets the class attribute as a string
  if (typeof className === 'string') {
    el.className = className
  }
  // Sets the class attribute as an array
  if (Array.isArray(className)) {
    el.classList.add(...className)
  }
}
```

SETTING THE STYLE ATTRIBUTES

SETTING THE STYLE ATTRIBUTES

- Instance of `CSSStyleDeclaration` object

SETTING THE STYLE ATTRIBUTES

- Instance of `CSSStyleDeclaration` object

```
element.style.color = 'red'  
element.style.fontFamily = 'Georgia'
```

SETTING THE STYLE ATTRIBUTES

- Instance of `CSSStyleDeclaration` object

```
element.style.color = 'red'  
element.style.fontFamily = 'Georgia'
```

If applied to a `<p>` element:

SETTING THE STYLE ATTRIBUTES

- Instance of `CSSStyleDeclaration` object

```
element.style.color = 'red'  
element.style.fontFamily = 'Georgia'
```

If applied to a `<p>` element:

```
<p style="color: red; font-family: Georgia;"></p>
```

```
export function setStyle(el, name, value) {  
  el.style[name] = value  
}  
  
export function removeStyle(el, name) {  
  el.style[name] = null  
}
```

SETTING THE REST OF THE ATTRIBUTES

- Attributes are set using assignment to properties
- If the attribute shouldn't be present:
 - set the value to `null`
 - use the `removeAttribute()` method

```
export function setAttribute(el, name, value) {  
  if (value == null) {  
    removeAttribute(el, name)  
  } else if (name.startsWith('data-')) {  
    el.setAttribute(name, value)  
  } else {  
    el[name] = value  
  }  
}
```

```
export function removeAttribute(el, name) {  
  el[name] = null
```



```
el.removeAttribute(name)  
}
```

MOUNT DOM EXAMPLE

If the view is given as:

```
const vdom = h('section', {} [  
  h('h1', {}, ['My Blog']),  
  h('p', {}, ['Welcome to my blog!'])  
)  
mountDOM(vdom, document.body)
```

the resulting HTML:

```
<body>  
  <section>  
    <h1>My Blog</h1>  
    <p>Welcome to my blog!</p>  
  </section>  
</body>
```

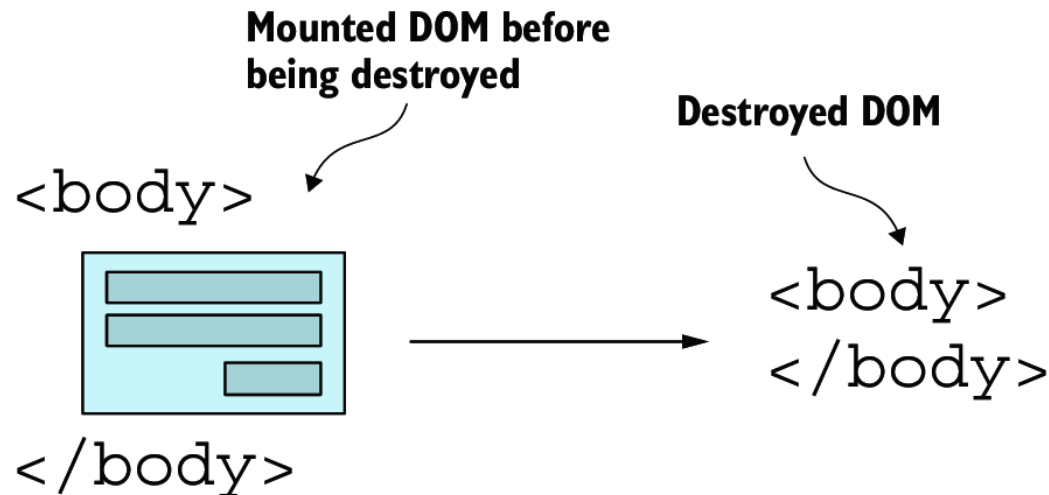
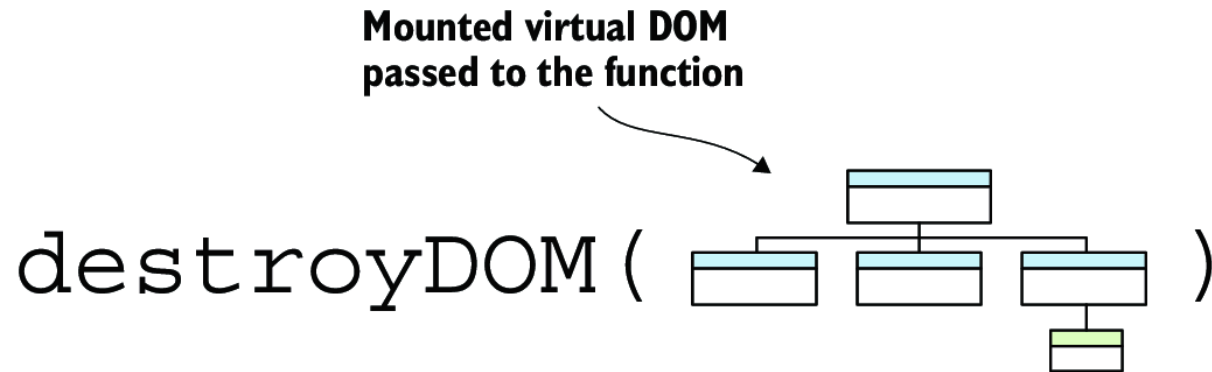
DESTROYING THE DOM

DESTROYING THE DOM

- The elements are removed from the document

DESTROYING THE DOM

- The elements are removed from the document



Considerations:

Considerations:

Text node

Considerations:

Text node

Remove the text node from its parent element, using the `remove()` method.

Considerations:

Text node

Remove the text node from its parent element, using the `remove()` method.

Fragment node

Considerations:

Text node

Remove the text node from its parent element, using the `remove ( )` method.

Fragment node

Remove each of its children from the parent element (which, if you recall, is referenced in the `e1` property of the fragment virtual node).

Considerations:

Text node

Remove the text node from its parent element, using the `remove()` method.

Fragment node

Remove each of its children from the parent element (which, if you recall, is referenced in the `e1` property of the fragment virtual node).

Element node

Considerations:

Text node

Remove the text node from its parent element, using the `remove()` method.

Fragment node

Remove each of its children from the parent element (which, if you recall, is referenced in the `e1` property of the fragment virtual node).

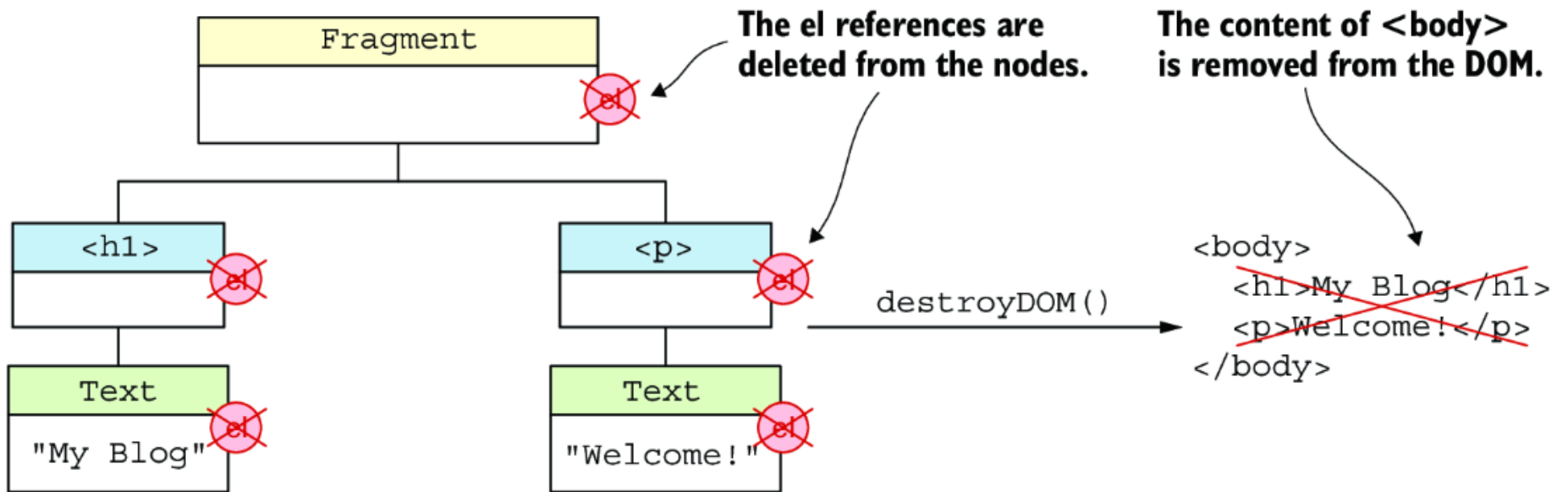
Element node

Do the two preceding things and remove the event listeners from the element.

- remove the `el` property from the virtual node,
- for an element node, you also remove the `listeners` property.
- If a virtual node doesn't have an `el` property, you can safely assume that it's not mounted to the real DOM and therefore can't be destroyed.

```
import { removeEventListeners } from './events'
import { DOM_TYPES } from './h'

export function destroyDOM(vdom) {
  const { type } = vdom
  switch (type) {
    case DOM_TYPES.TEXT: {
      removeTextNode(vdom)
      break
    }
    case DOM_TYPES.ELEMENT: {
      removeElementNode(vdom)
      break
    }
    case DOM_TYPES.FRAGMENT: {
      removeFragmentNodes(vdom)
      break
    }
    default: {
      throw new Error(`Can't destroy DOM of type: ${type}`)
    }
  }
  delete vdom.el
}
```



DESTROYING TEXT NODES

```
function removeTextNode(vdom) {  
  const { el } = vdom  
  el.remove()  
}
```


DESTROYING AN ELEMENT

- Start by removing it from the DOM
- Recursively destroy the children
- Remove the event listeners and delete the `listeners` property

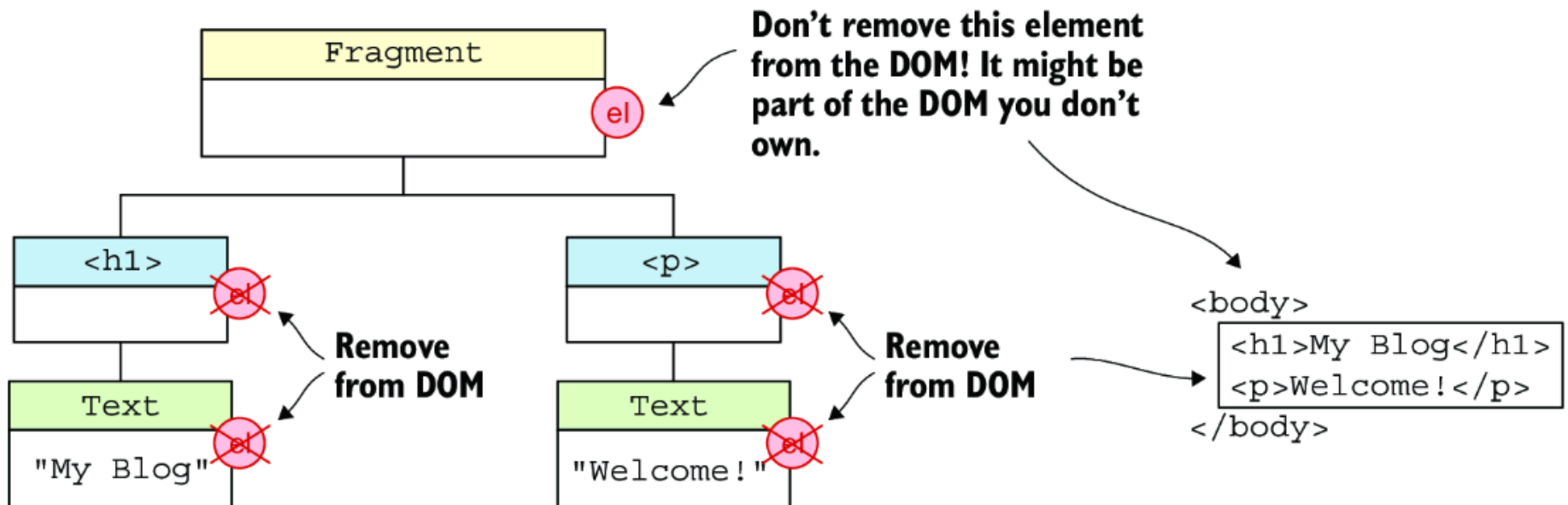
```
function removeElementNode(vdom) {  
  const { el, children, listeners } = vdom  
  
  el.remove()  
  children.forEach(destroyDOM)  
  
  if (listeners) {  
    removeEventListeners(listeners, el)  
    delete vdom.listeners  
  }  
}
```

Lets write the `removeEventListeners()` function

```
// FILE: src/events.js
export function removeEventListeners(listeners = {}, el) {
  Object.entries(listeners).forEach(([eventName, handler]) => {
    el.removeEventListener(eventName, handler)
  })
}
```

DESTROYING A FRAGMENT NODE

- call the `destroyDOM()` function for each child
- do not remove the `el` referenced in the fragment *vnode*



```
function removeFragmentNodes(vdom) {  
  const { children } = vdom  
  children.forEach(destroyDOM)  
}
```